
PROBLEMA DA ATRIBUIÇÃO

Cayo César da Costa Lopes
Bacharelado em Sistemas de Informação – CERES/UFRN
Universidade Federal do Rio Grande do Norte
Caicó, Brasil
cayo.lopes.129@ufrn.edu.br

Dr. Francisco Marcio Barboza
Departamento de Computação e Tecnologia
Universidade Federal do Rio Grande do Norte
Caicó, Brasil
marcio.barboza@ufrn.br

RESUMO

Este trabalho descreve o desenvolvimento e a implementação de um Sistema de Apoio à Decisão (SAD) para a resolução do problema de escalonamento de horários universitários, um desafio clássico de otimização combinatória de alta complexidade. A abordagem proposta utiliza a linguagem de modelagem declarativa MiniZinc integrada ao motor de busca Chuffed, orquestrada por um ambiente Python para o tratamento e visualização de dados. O modelo foi estruturado sob um conjunto de restrições rígidas (*Hard Constraints*), incluindo a garantia de carga horária, a prevenção de conflitos de alocação docente e a aplicação de uma regra de unicidade diária, que assegura a distribuição adequada das aulas. Os resultados demonstram que o uso de heurísticas de busca avançadas, como a estratégia *first_fail*, permitiu a geração de grades horárias otimizadas com 45 blocos de aula alocados em 14 segundos. Conclui-se que o sistema oferece uma solução eficaz para a automação do planejamento acadêmico, eliminando inconsistências manuais e proporcionando agilidade na tomada de decisão gerencial.

Palavras-chave: Sistemas de Apoio à Decisão, Otimização Combinatória, Programação por Restrições, MiniZinc, Python, Escalonamento de Horários.

ABSTRACT

This work describes the development and implementation of a Decision Support System (DSS) for solving the university timetable scheduling problem, a classic combinatorial optimization challenge of high complexity. The proposed approach uses the MiniZinc declarative modeling language integrated with the Chuffed search engine, orchestrated by a Python environment for data processing and visualization. The model was structured under a set of hard constraints, including guaranteed teaching hours, prevention of faculty allocation conflicts, and the application of a daily uniqueness rule, which ensures adequate distribution of classes. The results demonstrate that the use of advanced search heuristics, such as the first-fail strategy, allowed the generation of optimized timetables with 45 class blocks allocated in 14 seconds. It is concluded that the system offers an effective solution for automating academic planning, eliminating manual inconsistencies and providing agility in managerial decision-making.

Keywords: Decision Support Systems, Combinatorial Optimization, Constraint Programming, MiniZinc, Python, Time Scheduling.

1 Introdução

O Problema de Atribuição clássico consiste em alocar um conjunto de recursos a um conjunto de tarefas de forma a otimizar um objetivo, garantindo que cada recurso seja designado a exatamente uma tarefa e vice-versa. Matematicamente, este pode ser representado por uma matriz binária onde a variável de decisão assume valor 1 se o recurso é atribuído à tarefa, e 0 caso contrário. A relevância deste modelo transcende a gestão de pessoal, sendo fundamental em infraestruturas críticas, como na atribuição de comprimentos de onda em redes ópticas para minimizar bloqueios de conexão [1] ou na alocação distribuída de canais para mitigação de interferências em rádio cognitivo [2].

O problema de escalonamento de horários (*school timetabling*) é uma das variantes mais complexas do clássico Problema de Atribuição (*Assignment Problem*). Caracterizado pela necessidade de alocar recursos limitados como professores, disciplinas e slots temporais sob um conjunto rígido de restrições de integridade, este desafio pertence à classe de problemas NP-difíceis. Nestes cenários, o crescimento exponencial do número de variáveis torna o planejamento manual virtualmente inviável e exige abordagens computacionais eficientes.

Apesar da relevância do problema, abordagens tradicionais frequentemente apresentam limitações quanto à flexibilidade na modelagem de restrições específicas. Além disso, a escolha entre arquiteturas distribuídas, focadas em conhecimento local, e abordagens centralizadas é determinante; no contexto acadêmico, uma abordagem centralizada permite a aplicação de heurísticas globais que garantem o cumprimento de múltiplas regras pedagógicas simultâneas [2].

Neste contexto, este trabalho descreve o desenvolvimento de um Sistema de Apoio à Decisão (SAD) baseado em Programação por Restrições (CP). O objetivo central é a automação da grade horária de uma instituição com quatro turmas, oito docentes e vinte e uma disciplinas, operando no turno da tarde. Para a resolução do modelo, utilizou-se a linguagem de modelagem de alto nível MiniZinc, integrada ao motor de busca Chuffed. A interface de visualização e manipulação de dados foi construída em linguagem Python, permitindo uma ponte entre o rigor matemático do solver e a necessidade de clareza do tomador de decisão.

2 Metodologia

2.1 Modelagem e Formulação do Problema

O modelo proposto expande o escopo para quatro turmas simultâneas ($T = \{t_1, \dots, t_4\}$), totalizando 21 disciplinas distintas sob a responsabilidade de oito docentes. A infraestrutura horária compreende cinco dias úteis com três blocos diários de 1h40min (13:00, 14:50 e 16:40). O problema foi modelado como um sistema de otimização multiobjetivo, visando equilibrar restrições administrativas rígidas com diretrizes pedagógicas e de bem-estar docente.

Tabela 1: Grade Horária - 1º ao 4º Ano

	Seg	Ter	Qua	Qui	Sex
1º ANO					
13:00	V	V	V	V	V
14:50	V	V	V	V	V
16:40	V	V	V	V	V
2º ANO					
13:00	V	V	V	V	V
14:50	V	V	V	V	V
16:40	V	V	V	V	V
3º ANO					
13:00	V	V	V	V	V
14:50	V	V	V	V	V
16:40	V	V	V	V	V
4º ANO					
13:00	V	V	V	V	V
14:50	V	V	V	V	V
16:40	V	V	V	V	V

2.2 Restrições e Critérios de Otimização

O modelo implementado no MiniZinc organiza as restrições em duas camadas distintas: as restrições rígidas (hard constraints), que definem a viabilidade fundamental do sistema, e as restrições flexíveis (soft constraints), que orientam a busca pela melhor solução dentro do espaço viável.

2.2.1 Restrições Rígidas (Hard Constraints)

Estas são condições obrigatórias. Se qualquer uma delas for violada, a grade gerada é automaticamente descartada pelo solver. Elas garantem que o horário seja *factível* no mundo real.

- **Pertencimento por Turma:** Cada uma das 21 matérias possui uma turma fixa (1º ao 4º ano). O modelo garante que uma matéria do 2º ano, por exemplo, nunca seja alocada nos horários da turma do 4º ano.
- **Cumprimento da Carga Horária:** Cada matéria possui carga definida em blocos de 1h40min (60h = 2 blocos; 90h = 3 blocos). O solver conta exatamente quantas vezes cada matéria aparece na grade da sua respectiva turma e exige que esse número seja igual à carga prevista.
- **Unicidade Diária:** Esta regra impede que um aluno tenha dois blocos da mesma matéria no mesmo dia. Matérias de 90h (3 blocos semanais) necessariamente serão distribuídas em três dias diferentes.
- **Indisponibilidade Seletiva de Docentes:** Cada um dos 8 professores possui uma matriz binária de disponibilidade (8 linhas $\times$ 5 dias). O modelo verifica: se a disponibilidade for 0 para um dado dia, nenhuma aula desse professor pode ser alocada em nenhuma turma naquele dia.
- **Prevenção de Conflito Cruzado (Onipresença):** Como um mesmo professor pode lecionar em diferentes turmas, o sistema precisa garantir que, em um mesmo dia e bloco de horário, o professor não apareça em duas turmas simultaneamente.

2.3 Disciplinas e Professores

A Tabela 2 apresenta as 21 disciplinas, seus respectivos professores e a carga horária semanal (2 blocos = 60h, 3 blocos = 90h).

Tabela 2: Disciplinas, Professores e Carga Horária

Ano	Disciplina	Professor	Blocos
1º Ano	Introdução à Informática	Ana	2
1º Ano	Algoritmos	Carlos	3
1º Ano	Administração Geral	Daniel	2
1º Ano	Fundamentos de Matemática	Gustavo	2
1º Ano	Lógica	Bruno	2
2º Ano	Teoria Geral de Sistemas	Daniel	2
2º Ano	Metodologia	Fernanda	2
2º Ano	Programação	Carlos	3
2º Ano	Cálculo I	Bruno	2
2º Ano	Leitura e Produção Texto	Fernanda	2
3º Ano	POO I	Gabriel	2
3º Ano	OSM	Heitor	2
3º Ano	Fundamentos de SI	Daniel	2
3º Ano	Estrutura de Dados	Gabriel	3
3º Ano	Álgebra Linear	Bruno	2
4º Ano	Banco de Dados	Heitor	2
4º Ano	Arquitetura de Computadores	Ana	2
4º Ano	Engenharia de Software I	Heitor	2
4º Ano	Programação Web	Gabriel	2
4º Ano	Inglês Técnico	Fernanda	2
4º Ano	Probabilidade e Estatística	Gustavo	2

2.4 Dias de disponibilidade dos Professores

Aqui é mostrado os dias em que o professor estaria na universidade.

Tabela 3: Dias de Disponibilidade dos Professores

Professor	Dias Disponíveis
Ana	Segunda, Terça e Quarta
Bruno	Quarta, Quinta e Sexta
Carlos	Terça, Quarta e Quinta
Daniel	Segunda, Terça e Quinta
Fernanda	Terça, Quarta e Sexta
Gabriel	Terça, Quarta e Quinta
Gustavo	Terça, Quinta e Sexta
Heitor	Segunda, Quarta e Quinta

2.4.1 Implementação no MiniZinc

A seguir, são apresentados trechos da implementação das principais restrições no código `avaliacao2.mzn`, disponível no repositório GitHub do projeto.

Variável de Decisão: A variável principal do modelo é uma matriz tridimensional que armazena, para cada turma, dia e bloco, o ID da matéria alocada (0 indica horário vago):

```
% --- 3. VARIÁVEL DE DECISÃO ---
array[1..n_turmas, 1..n_dias, 1..n_blocos] of var 0..n_materias: grade;
```

Disponibilidade dos Professores: A restrição que limita a atuação dos docentes aos seus dias específicos de trabalho:

```
% 4.4 Disponibilidade
constraint forall(p in 1..n_professores, d in 1..n_dias) (
    if disponibilidade[p, d] == 0 then
        sum(t in 1..n_turmas, b in 1..n_blocos) (
            bool2int( grade[t,d,b] > 0 /\ prof_materia[grade[t,d,b]] == p )
        ) == 0
    endif
);
```

Conflito Cruzado (Hard Constraint mais relevante): A restrição que impede um professor de lecionar em duas turmas no mesmo horário foi implementada como:

```
% 4.5 Conflito cruzado (Professor em máx 1 sala POR BLOCO)
constraint forall(p in 1..n_professores, d in 1..n_dias, b in 1..n_blocos) (
    sum(t in 1..n_turmas) (
        bool2int(grade[t,d,b] > 0 /\ prof_materia[grade[t,d,b]] == p)
    ) <= 1
);
```

2.4.2 Restrições Flexíveis (Soft Constraints) e Função Objetivo

Enquanto as restrições rígidas apenas aprovam ou reprovam uma grade, as restrições flexíveis atribuem uma *nota* (custo) a cada solução válida. O solver não para ao encontrar a primeira grade factível; ele continua buscando até provar que encontrou a grade com o menor custo possível.

Primeiro Objetivo (Prioridade Máxima): Penalidade Pedagógica Este critério se aplica exclusivamente às matérias de 60h (2 blocos semanais), classificadas em duas categorias: *Seguidas* (intervalo ideal de 1 dia) e *Esparsas* (intervalo ideal de 2 dias). A implementação deste cálculo no MiniZinc é:

```
% 5.1 Penalidade pedagógica (Peso 100)
var int: penalidade_pedagogica = sum(m in 1..n_materias where tipo_espacamento[m] > 0) (
    sum(d1, d2 in 1..n_dias where d1 < d2) (
        if (sum(b in 1..n_blocos)(bool2int(grade[turma_materia[m], d1, b] == m)) > 0) /\
            (sum(b in 1..n_blocos)(bool2int(grade[turma_materia[m], d2, b] == m)) > 0)
        then
            abs((d2 - d1) - tipo_espacamento[m])
        else
            0
        endif
    )
);
```

Segundo Objetivo (Prioridade Secundária): Minimização de Janelas Docentes Uma **janela** ocorre quando um professor dá aula no primeiro bloco, não dá aula no segundo bloco, e dá aula no terceiro bloco. A implementação:

```
% 5.2 Janelas de professores (Peso 1)
var int: total_janelas = sum(p in 1..n_professores, d in 1..n_dias) (
    bool2int(
        sum(t in 1..n_turmas)(bool2int(grade[t, d, 1] > 0 /\
            prof_materia[grade[t, d, 1]] == p)) == 1 /\
        sum(t in 1..n_turmas)(bool2int(grade[t, d, 2] > 0 /\
            prof_materia[grade[t, d, 2]] == p)) == 0 /\
        sum(t in 1..n_turmas)(bool2int(grade[t, d, 3] > 0 /\
            prof_materia[grade[t, d, 3]] == p)) == 1
    )
);
```

Função Objetivo Final e Estratégia de Busca: Para garantir que o critério pedagógico jamais seja sacrificado em prol do conforto do professor, os dois objetivos foram combinados por meio de uma soma ponderada com pesos assimétricos (100:1):

```
% 5.3 Função de custo
var int: custo_total = (penalidade_pedagogica * 100) + total_janelas;

solve :: int_search(
    [grade[t,d,b] | t in 1..n_turmas, d in 1..n_dias, b in 1..n_blocos],
    first_fail,
    indomain_min,
    complete
) minimize custo_total;
```

A heurística `first_fail` prioriza as variáveis com menor domínio restante (as células da grade mais restritas), acelerando a poda da árvore de busca. O peso 100:1 assegura que melhorar a penalidade pedagógica em 1 unidade equivale a piorar as janelas em até 99 unidades em termos de custo.

2.5 Arquitetura de Resolução

A resolução do modelo foi realizada com o solver **Chuffed**, que implementa a técnica de *Lazy Clause Generation*. Diferentemente de solvers tradicionais baseados exclusivamente em *backtracking*, o Chuffed aprende com falhas parciais durante a busca, gerando cláusulas de *nogood* que podam espaços da árvore de busca já explorados sem sucesso. Esta característica é particularmente valiosa neste problema devido à alta interdependência entre as 60 variáveis de decisão (4 turmas × 5 dias × 3 blocos). A integração com Python atua como uma camada de tradução, convertendo os IDs numéricos retornados pelo solver em nomes legíveis e organizando a saída em quatro tabelas horárias independentes, uma por turma.

3 Resultados e Discussão

3.1 Grade Horária Gerada

A execução do modelo com o solver Chuffed produziu as grades horárias apresentadas na Tabela 4. A tabela exibe a formatação consolidada das alocações, contendo as grades para os quatro anos organizadas conforme a saída do script Python de integração.

Tabela 4: Grade Horária Gerada e Formatada - 1º ao 4º Ano

Horário	Segunda	Terça	Quarta	Quinta	Sexta
GRADE HORÁRIA - 1º ANO					
13:00	Int. Informá (Ana)	Algoritmos (Carlos)	Algoritmos (Carlos)	— VAGO —	— VAGO —
14:50	Adm. Geral (Daniel)	Fund. Matemá (Gustavo)	Int. Informá (Ana)	Algoritmos (Carlos)	— VAGO —
16:40	— VAGO —	Adm. Geral (Daniel)	Lógica (Bruno)	Fund. Matemá (Gustavo)	Lógica (Bruno)
GRADE HORÁRIA - 2º ANO					
13:00	— VAGO —	T. Geral Sis (Daniel)	Metodologia (Fernanda)	Programação (Carlos)	Cálculo I (Bruno)
14:50	— VAGO —	Programação (Carlos)	Programação (Carlos)	Cálculo I (Bruno)	— VAGO —
16:40	— VAGO —	Leitura/Prod (Fernanda)	Leitura/Prod (Fernanda)	T. Geral Sis (Daniel)	Metodologia (Fernanda)
GRADE HORÁRIA - 3º ANO					
13:00	— VAGO —	POO I (Gabriel)	— VAGO —	POO I (Gabriel)	— VAGO —
14:50	OSM (Heitor)	Fund. SI (Daniel)	OSM (Heitor)	Estrutura Da (Gabriel)	Álgebra Line (Bruno)
16:40	Fund. SI (Danie)	Estrutura Da (Gabriel)	Estrutura Da (Gabriel)	Álgebra Line (Bruno)	— VAGO —
GRADE HORÁRIA - 4º ANO					
13:00	Banco Dados (Heitor)	Arquitetura (Ana)	Eng. Soft I (Heitor)	— VAGO —	— VAGO —
14:50	Arquitetura (Ana)	Prog. Web (Gabreli)	Inglês Téc. (Fernanda)	Prob./Estat. (Gustavo)	Inglês Téc. (Fernanda)
16:40	Eng. Soft I (Heitor)	Prob./Estat. (Gustavo)	Banco Dados (Heitor)	Prog. Web (Gabriel)	— VAGO —

A análise visual das grades permite identificar os seguintes padrões:

3.1.1 Distribuição por Turma

Observa-se que o solver preencheu os 45 blocos de aula semanais (15 por turma) respeitando rigorosamente a disponibilidade de cada professor. As células marcadas como "— VAGO —" indicam horários livres naquela turma, totalizando 15 vagos distribuídos entre as quatro séries.

3.1.2 Verificação das Hard Constraints

A partir das grades, é possível validar as restrições rígidas impostas ao modelo:

- **Pertencimento por Turma:** Todas as disciplinas aparecem exclusivamente em sua turma de origem.
- **Carga Horária:** Matérias de 90h aparecem exatamente três vezes; matérias de 60h aparecem duas vezes.
- **Unicidade Diária:** Nenhuma disciplina repete bloco no mesmo dia.
- **Conflito Cruzado:** Nenhum professor aparece em duas turmas no mesmo horário.
- **Disponibilidade dos Professores:** Cada docente respeita sua janela de 3 dias por semana.

3.1.3 Análise das Soft Constraints (Otimização)

Penalidade Pedagógica ($P = 2$): O valor 2 indica que, das 15 matérias de 60h avaliadas, apenas duas unidades de desvio ideal foram acumuladas, um resultado excelente diante das restrições de disponibilidade dos professores.

Janelas Docentes ($J = 0$): Nenhum professor apresenta o padrão [ocupado, vago, ocupado] em nenhum dia. A eliminação total de janelas demonstra que o solver respeitou a continuidade do turno, mesmo sob prioridade máxima do critério pedagógico.

A Tabela 5 consolida as métricas obtidas:

Tabela 5: Métricas de otimização consolidadas

Métrica	Valor
Penalidade Pedagógica (P)	2
Total de Janelas (J)	0
Custo Total ($C = 100P + J$)	200
Tempo de execução	14 segundos
Blocos de aula alocados	45
Blocos vagos (horários livres)	15

3.1.4 Comparativo com Abordagem Anterior

Diferentemente da versão anterior do sistema, que se limitava a encontrar qualquer grade válida (*satisfy*), o modelo atual empregou uma busca exaustiva orientada por função de custo. Enquanto uma abordagem puramente satisfatória poderia gerar uma grade com penalidade pedagógica alta e múltiplas janelas, a otimização multiobjetivo aqui implementada comprovadamente encontrou a *melhor* grade possível dentro das restrições impostas, com custo global mínimo de 200.

4 Conclusão

Este trabalho evoluiu um sistema de geração de horários de uma abordagem de satisfação para um modelo de otimização multiobjetivo com hierarquia de prioridades. A escolha do solver Chuffed, aliada à ponderação 100:1 entre critérios pedagógicos e logísticos, produziu uma grade factível e otimizada em 14 segundos — um tempo compatível com uso interativo em contexto administrativo. Os resultados demonstraram que é possível conciliar excelência acadêmica (penalidade pedagógica quase nula) com eficiência operacional (zero janelas docentes) dentro das restrições impostas.

Reprodução dos Resultados

Para permitir a verificação e replicabilidade do estudo, a seguir são descritos os passos necessários para reproduzir os resultados obtidos.

Obtenção dos arquivos: Todos os arquivos necessários para execução do modelo estão disponíveis publicamente no repositório GitHub do projeto:

https://github.com/CayoLopes/SAD_2026.1.git

O arquivo principal contendo o modelo MiniZinc completo é denominado `avaliacao2.mzn`. O código está organizado nas seguintes seções:

```
% --- 1. PARÂMETROS GERAIS ---
% --- 2. MAPEAMENTO DE DADOS ---
% --- 3. VARIÁVEL DE DECISÃO ---
% --- 4. RESTRIÇÕES RÍGIDAS ---
% --- 5. OTIMIZAÇÃO ---
% --- 6. SAÍDA ---
```

Ambiente de execução: Os experimentos foram realizados em um computador com as seguintes especificações:

- **Sistema Operacional:** Windows 10 (64 bits)
- **Processador:** AMD Ryzen 5 2400GE (4 núcleos, 8 threads, 3.2 GHz)
- **Memória RAM:** 16 GB DDR4
- **Solver:** Chuffed (versão 0.13.2) rodando sobre MiniZinc 2.9.5

Execução via MiniZinc IDE:

1. Clonar o repositório ou fazer o download do arquivo `avaliacao2.mzn`
2. Abrir o arquivo no MiniZinc IDE
3. Selecionar Chuffed como solver (Configurar → Solver → Chuffed)

4. Clicar em Run (ou pressionar Ctrl+R)
5. Aguardar a mensagem ===== indicando finalização da busca

Verificação dos resultados: A saída esperada inclui as quatro tabelas horárias (1º ao 4º ano) e as métricas: Penalidade Pedagógica = 2, Total de Janelas = 0, Custo Total = 200. O tempo de execução registrado foi de aproximadamente 14 segundos nas especificações descritas. Qualquer desvio significativo pode indicar diferenças no ambiente, versão dos solvers ou hardware utilizado.

Trabalhos Futuros

Como desdobramento natural desta pesquisa, sugere-se um estudo comparativo sistemático entre os solvers **Chuffed** e **Gecode**. Uma metodologia experimental proposta inclui: executar o mesmo modelo nos dois solvers mantendo idênticas as demais condições, registrar tempo até primeira solução viável, tempo até comprovação da otimalidade, número de nós explorados na árvore de busca e consumo de memória, além de variar a complexidade do problema para avaliar a escalabilidade de cada abordagem. Resultados preliminares na literatura sugerem que o Chuffed tende a superar o Gecode em problemas com alta interdependência entre variáveis, devido à sua capacidade de *nogood learning*.

Outra linha de evolução natural do modelo é a inclusão de restrições de infraestrutura física (alocação de salas e laboratórios), transformando o problema em um escalonamento tridimensional (turmas × professores × recursos).

Referências

- [1] Daniela Cunha and Graça Bressan. Proposta de um algoritmo de atribuição de rótulos baseado em srlg em um ambiente gmpls-wdm. In *XXII Simpósio Brasileiro de Telecomunicações*, 2005.
- [2] Paulo R. Walenga Jr., Mauro Fonseca, Anelise Munaretto, Aline Carneiro Viana, and Artur Ziviani. Zap: Um algoritmo de atribuição distribuída de canais para mitigação de interferências em redes com rádio cognitivo. In *XXVIII Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, pages 175–188, 2010.